

Making a Class-Conditioned Diffusion Model

In this notebook we're going to illustrate one way to add conditioning information to a diffusion model. Specifically, we'll train a class-conditioned diffusion model on MNIST following on from the ['from-scratch' example in Unit 1](#), where we can specify which digit we'd like the model to generate at inference time.

As mentioned in the introduction to this unit, this is just one of many ways we could add additional conditioning information to a diffusion model, and has been chosen for its relative simplicity. Just like the 'from-scratch' notebook in Unit 1, this notebook is mostly for illustrative purposes and you can safely skip it if you'd like.

Setup and Data Prep

```
In [1]: %pip install -q diffusers
```

```
In [2]: import torch
import torchvision
from torch import nn
from torch.nn import functional as F
from torch.utils.data import DataLoader
from diffusers import DDPMSScheduler, UNet2DModel
from matplotlib import pyplot as plt
from tqdm.auto import tqdm

device = 'mps' if torch.backends.mps.is_available() else 'cuda' if torch.cuda.is_available() else 'cpu'
print(f'Using device: {device}')
```

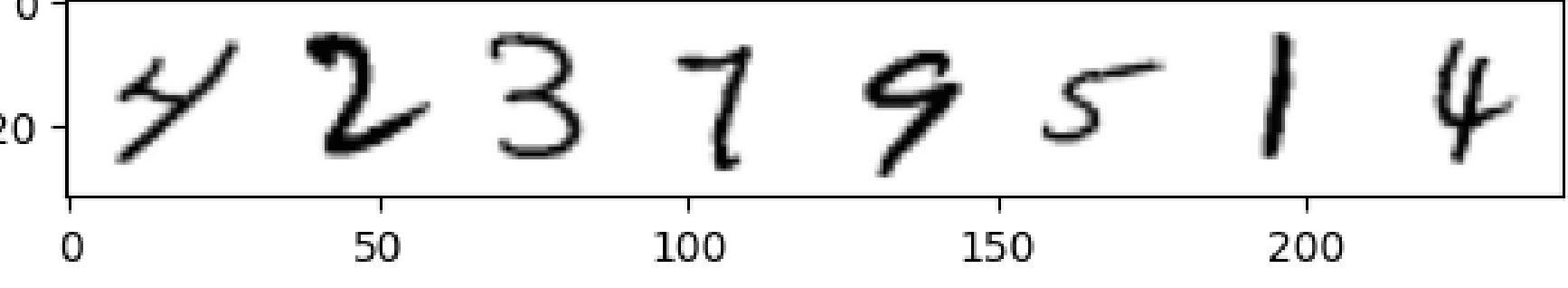
Flax classes are deprecated and will be removed in Diffusers v1.0.0. We recommend migrating to PyTorch classes or pinning your version of Diffusers.
Flax classes are deprecated and will be removed in Diffusers v1.0.0. We recommend migrating to PyTorch classes or pinning your version of Diffusers.

```
In [3]: # Load the dataset
dataset = torchvision.datasets.MNIST(root="mnist/", train=True, download=True, transform=torchvision.transforms.ToTensor())
```

```
# Feed it into a dataloader (batch size 8 here just for demo)
train_dataloader = DataLoader(dataset, batch_size=8, shuffle=True)

# View some examples
x, y = next(iter(train_dataloader))
print('Input shape:', x.shape)
print('Labels:', y)
plt.imshow(torchvision.utils.make_grid(x)[0], cmap='Greys');
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 18.4MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 524kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.77MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 12.8MB/s]
Input shape: torch.Size([8, 1, 28, 28])
Labels: tensor([4, 2, 3, 7, 9, 5, 1, 4])
```



Creating a Class-Conditioned UNet

The way we'll feed in the class conditioning is as follows:

- Create a standard `UNet2DModel` with some additional input channels
- Map the class label to a learned vector of shape `(class_emb_size)` via an embedding layer
- Concatenate this information as extra channels for the internal UNet input with `net_input = torch.cat((x, class_cond), 1)`
- Feed this `net_input` (which has `(class_emb_size+1)` channels in total) into the UNet to get the final prediction

In this example I've set the `class_emb_size` to 4, but this is completely arbitrary and you could explore having it size 1 (to see if it still works), size 10 (to match the number of classes), or replacing the learned `nn.Embedding` with a simple one-hot encoding of the class label directly.

This is what the implementation looks like:

```
In [4]: class ClassConditionedUnet(nn.Module):
def __init__(self, num_classes=10, class_emb_size=4):
    super().__init__()

    # The embedding layer will map the class label to a vector of size class_emb_size
    self.class_emb = nn.Embedding(num_classes, class_emb_size)

    # Self.model is an unconditional UNet with extra input channels to accept the conditioning information (the class embedding)
    self.model = UNet2DModel(
        sample_size=28, # the target image resolution
        in_channels=1 + class_emb_size, # Additional input channels for class cond.
        out_channels=1, # the number of output channels
        layers_per_block=2, # how many ResNet Layers to use per UNet block
        block_out_channels=(32, 64, 64),
        down_block_types=(
            "DownBlock2D", # a regular ResNet downsampling block
            "AttnDownBlock2D", # a ResNet downsampling block with spatial self-attention
            "AttnDownBlock2D",
        ),
        up_block_types=(
            "AttnUpBlock2D",
            "AttnUpBlock2D", # a ResNet upsampling block with spatial self-attention
            "UpBlock2D", # a regular ResNet upsampling block
        ),
    )

    # Our forward method now takes the class labels as an additional argument
    def forward(self, x, t, class_labels):
        # Shape of x:
        bs, ch, w, h = x.shape

        # class conditioning in right shape to add as additional input channels
        class_cond = self.class_emb(class_labels) # Map to embedding dimension
        class_cond = class_cond.view(bs, class_cond.shape[1], 1, 1).expand(bs, class_cond.shape[1], w, h)
        # x is shape (bs, 1, 28, 28) and class_cond is now (bs, 4, 28, 28)

        # Net input is now x and class_cond concatenated together along dimension 1
        net_input = torch.cat((x, class_cond), 1) # (bs, 5, 28, 28)

        # Feed this to the UNet alongside the timestep and return the prediction
        return self.model(net_input, t).sample # (bs, 1, 28, 28)
```

If any of the shapes or transforms are confusing, add in print statements to show the relevant shapes and check that they match your expectations. I've also annotated the shapes of some intermediate variables in the hopes of making things clearer.

Training and Sampling

Where previously we'd do something like `prediction = unet(x, t)` we'll now add the correct labels as a third argument (`prediction = unet(x, t, y)`) during training, and at inference we can pass whatever labels we want and if all goes well the model should generate images that match. `y` in this case is the labels of the MNIST digits, with values from 0 to 9.

The training loop is very similar to the [example from Unit 1](#). We're now predicting the noise (rather than the denoised image as in Unit 1) to match the objective expected by the default `DDPMScheduler` which we're using to add noise during training and to generate samples at inference time. Training takes a while - speeding this up could be a fun mini-project, but most of you can probably just skim the code (and indeed this whole notebook without running it since we're just illustrating an idea.

```
In [5]: # Create a scheduler
noise_scheduler = DDPMSScheduler(num_train_timesteps=1000, beta_schedule='squaredcos_cap_v2')
```

```
In [6]: ##markdown Training Loop (10 Epochs):
```

```
# Redefining the dataloader to set the batch size higher than the demo of 8
train_dataloader = DataLoader(dataset, batch_size=128, shuffle=True)

# How many runs through the data should we do?
n_epochs = 10

# Our network
net = ClassConditionedUnet().to(device)

# Our loss function
loss_fn = nn.MSELoss()

# The optimizer
opt = torch.optim.Adam(net.parameters(), lr=1e-3)

# Keeping a record of the losses for later viewing
losses = []

# The training loop
for epoch in range(n_epochs):
    for x, y in tqdm(train_dataloader):

        # Get some data and prepare the corrupted version
        x = x.to(device) * 2 - 1 # Data on the GPU (mapped to [-1, 1])
        y = y.to(device)
        noise = torch.randn_like(x)
        timesteps = torch.randint(0, 999, (x.shape[0],)).long().to(device)
        noisy_x = noise_scheduler.add_noise(x, noise, timesteps)

        # Get the model prediction
        pred = net(noisy_x, timesteps, y) # Note that we pass in the labels y

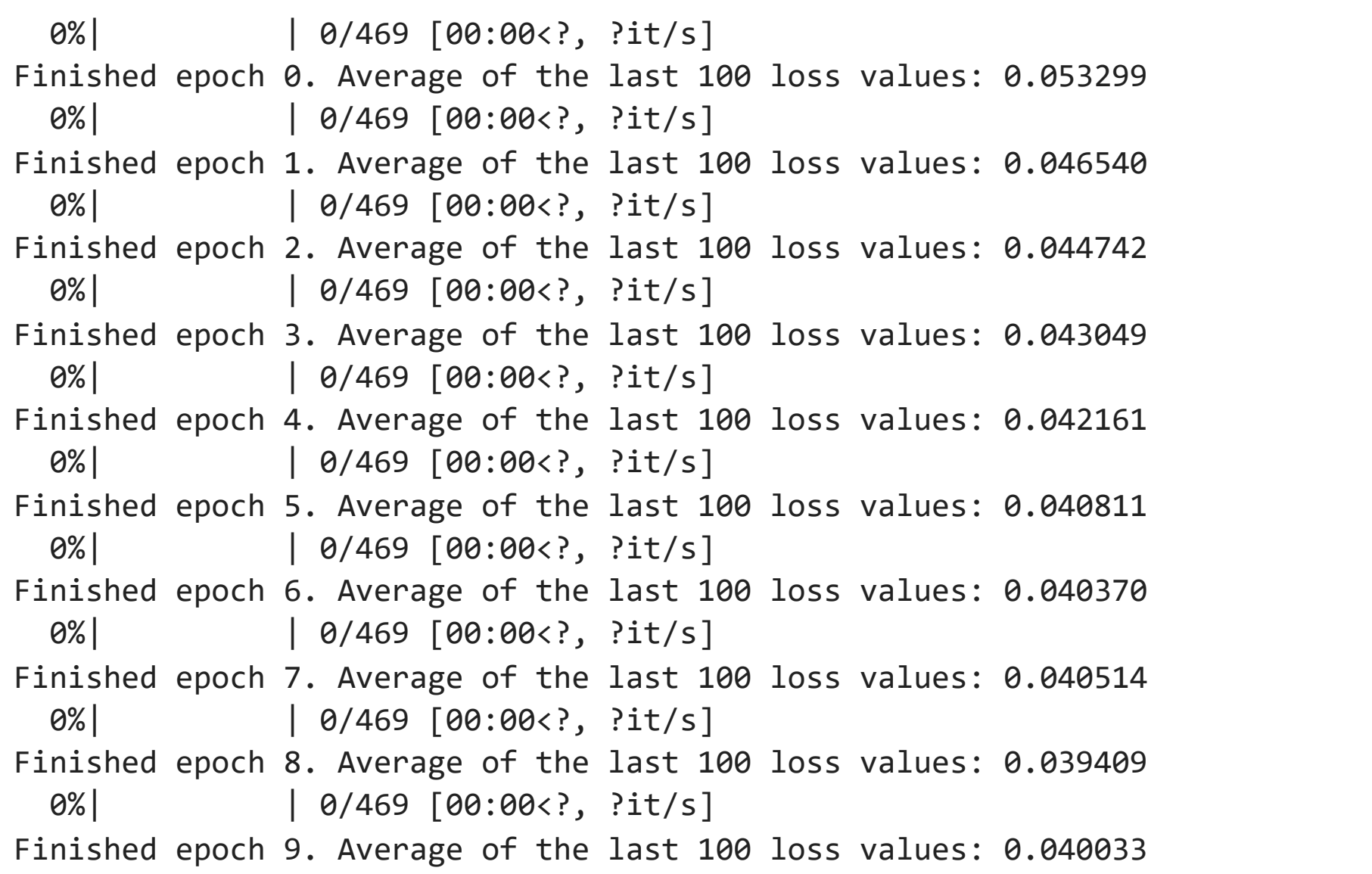
        # Calculate the loss
        loss = loss_fn(pred, noise) # How close is the output to the noise

        # Backprop and update the params:
        opt.zero_grad()
        loss.backward()
        opt.step()

        # Store the loss for later
        losses.append(loss.item())

# Print out the average of the last 100 loss values to get an idea of progress:
avg_loss = sum(losses[-100:])/100
print(f'Finished epoch {epoch}. Average of the last 100 loss values: {avg_loss:05f}')
```

```
# View the loss curve
plt.plot(losses)
```



Once training finishes, we can sample some images feeding in different labels as our conditioning:

```
In [7]: ##markdown Sampling some different digits:
```

```
# Prepare random x to start from, plus some desired labels y
x = torch.randn(80, 1, 28, 28).to(device)
y = torch.tensor([1]*8 for i in range(10))).flatten().to(device)

# Sampling loop
for i, t in tqdm(enumerate(noise_scheduler.timesteps)):

    # Get model pred
    with torch.no_grad():
        residual = net(x, t, y) # Again, note that we pass in our labels y

    # Update sample with step
    x = noise_scheduler.step(residual, t, x).prev_sample

# Show the results
fig, ax = plt.subplots(1, 1, figsize=(12, 12))
ax.imshow(torchvision.utils.make_grid(x.detach().cpu().clip(-1, 1), nrow=8)[0], cmap='Greys')
```

```
Out[7]: <matplotlib.image.AxesImage at 0x7b080e3418e0>
```



There we go! We can now have some control over what images are produced.

I hope you've enjoyed this example. As always, feel free to ask questions in the Discord.

```
In [ ]: # Exercise (optional): Try this with FashionMNIST. Tweak the learning rate, batch size and number of epochs.
# Can you get some decent-looking fashion images with less training time than the example above?
```